

Teaching Machines to Notice the Unexpected

LSTM-Based Anomaly Detection in Industrial Sensor Data

Machine Learning and Neural Networks | Postgraduate Tutorial | Author: Ameer Hamza

[GitHub](#)

Technique	Dataset	Task	Metric
LSTM	SKAB v0.9	Anomaly Detection	Precision/Recall/F1

Introduction & Motivation

Industrial systems deteriorate over time. Before catastrophic failure, a valve closes, shaft vibration increases, and motor current shifts. The traditional rule-based alarms mandate that a human expert make an exhaustive list of all possible failure modes. A better way: machine learning to know what normal is over time and flag deviations automatically.

In standard ML models, data points are considered independent. The readings from the sensor are not a fault that occurred 40 seconds ago, which will determine the current reading. Long-range temporal dependencies are best learned by LSTMs (Hochreiter and Schmidhuber, 1997), the obvious choice for this task.

What You Will Learn

1. The vanishing gradient problem, and precisely how LSTM gates resolve it.
2. Implementing and training a stacked LSTM with Keras for time-series prediction
3. How to use prediction error as a statistical anomaly signal
4. How to evaluate an anomaly detector using Precision, Recall, and F1-Score
5. How to critically analyze model failures and tune threshold sensitivity
6. Ethical implications of automated fault detection in industrial settings

The Dataset

The Skoltech Anomaly Benchmark (SKAB v0.9, Katser and Kozitsin, 2020) was collected from a real water circulation system at Skoltech, Russia. Eight sensors took measurements at one-second intervals; sensors were physically induced with anomalies (valve closures, shaft imbalance, cavitation), and every anomaly was exactly labelled with a second of occurrence, thus offering ground truth associated with real physical anomalies.

Table 1: SKAB Dataset Summary

Property	Detail
Source	Kaggle
File/Rate	35 CSV Experiment Files, 1 Reading/second
Sensors	accelerometers x2, current, pressure, temperature, thermocouple, voltage, flow rate
Label	anomaly column: 0 = normal, 1 = anomaly (point and collective types)

Exploratory Data Analysis

All eight sensors are plotted against time in Figure 1. The changes are best visible in the accelerometers' data within the anomaly window. The accelerometers are perfectly correlated ($r=1.00$), and they are negatively correlated with flow rate ($r=-0.68$) when the acceleration increases when a fault occurs as Shown in Figure 2. A violin plot confirms that the distribution of the data in the accelerometer changes greatly between normal and anomaly periods, as seen in Figure 3.

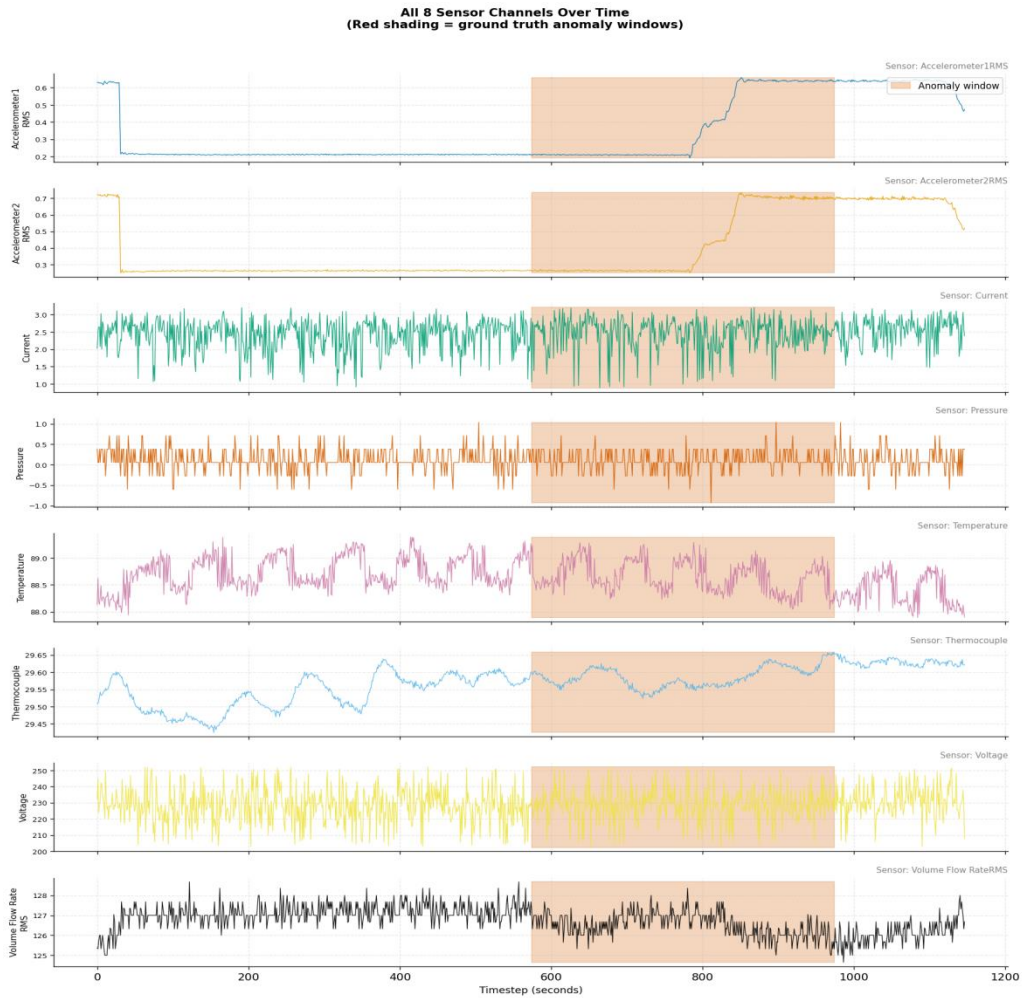


Figure 1: All 8 sensor channels over time. Red shading = ground truth anomaly window. Accelerometers show the most pronounced response

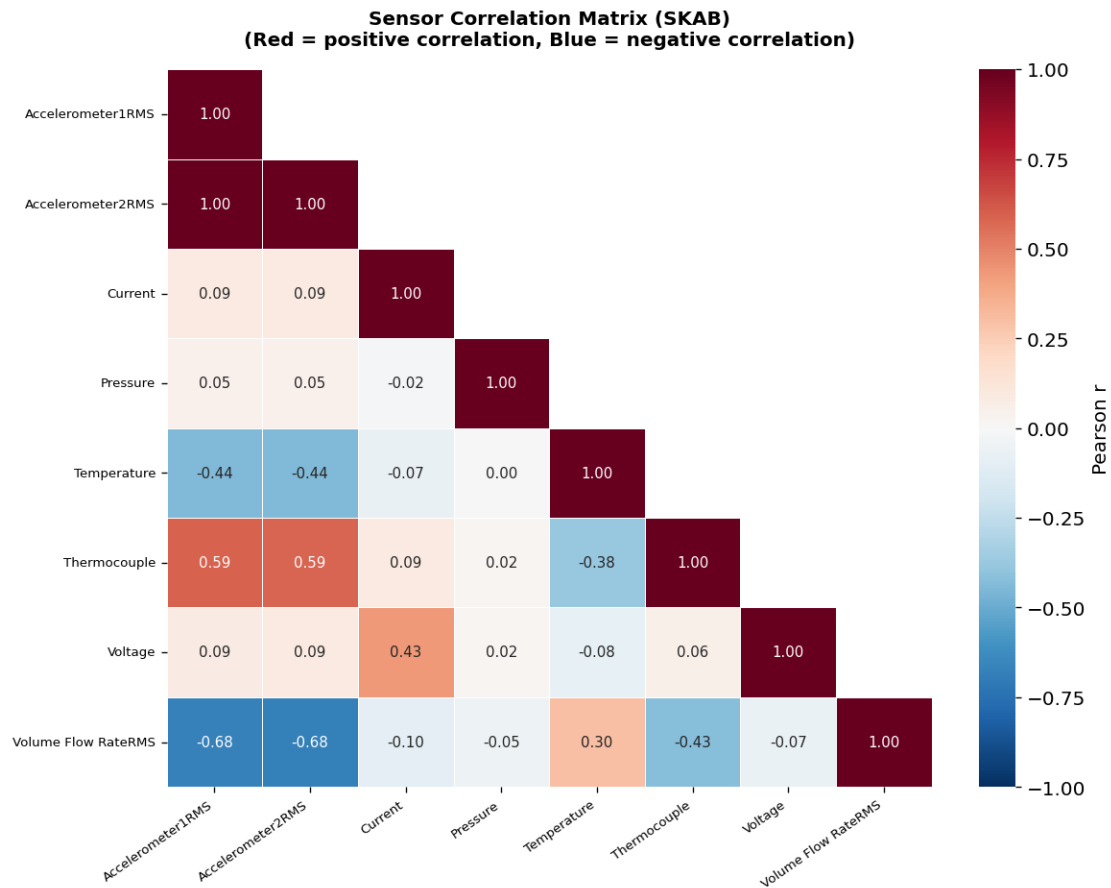


Figure 2: Sensor correlation matrix. Accelerometers perfectly correlated; flow rate negatively correlated with vibration ($r=-0.68$).

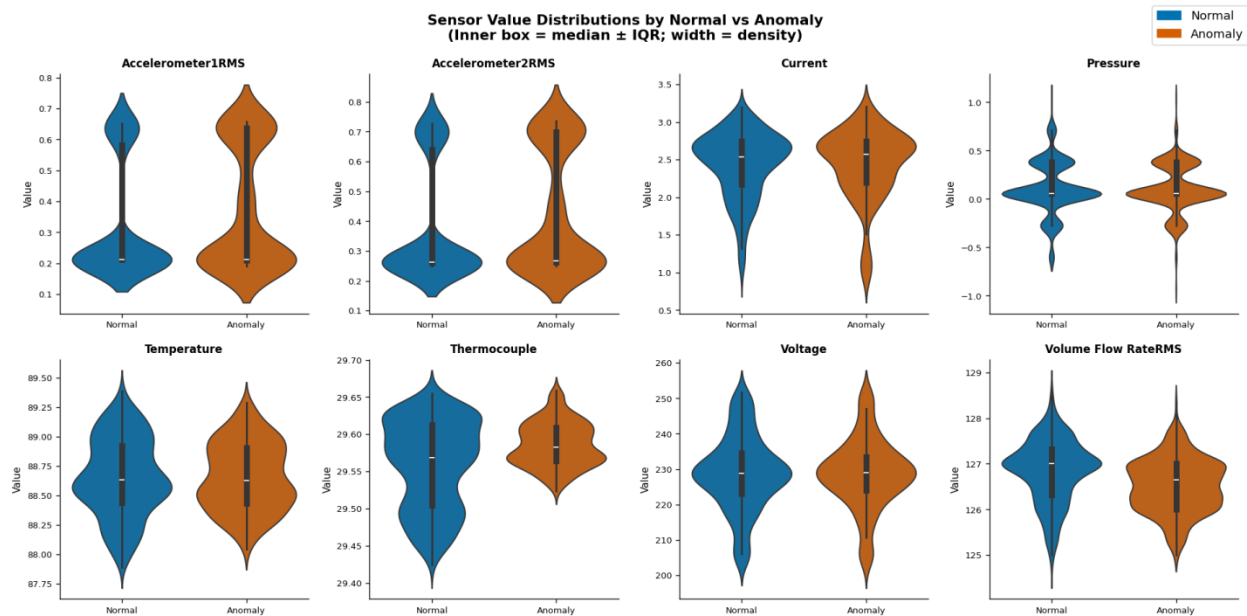


Figure 3: Sensor distributions Normal vs Anomaly. Accelerometers show the strongest shift

How LSTMs Work

The Vanishing Gradient Problem

Before the LSTM, Recurrent Neural Networks (RNNs) would take a sequence as a series of inputs, with the hidden state passed along one timestep at a time. The training involved backpropagating the gradients through each timestep in the sequence - Backpropagation Through Time (BPTT). The gradient at an early timestep is the product of the gradients of all the later timesteps. With these individual gradients being small (less than 1), this product will shrink exponentially towards zero, thus eliminating the vanishing gradient problem. It is possible to forget the context more than a few timesteps back. This is a basic problem when the LSTM is required to detect anomalies, such as 30 seconds of normal vibration followed by one of abnormal vibration, in which case the LSTM must remember that the vibration was normal 30 seconds ago to recognize the abnormal vibration as an anomaly.

LSTM Gates The Solution

LSTM cell (Hochreiter and Schmidhuber, 1997) adds a cell state, a long-term memory highway, to the hidden state. There are four learnable gates for information flow:

Table 2: LSTM gate equations and their roles.

Gate	Key Equation	Purpose
Forget	$f_t = \text{sigmoid}(W_f [h_{t-1}, x_t])$	What to discard from long-term memory
Input	$i_t = \text{sigmoid}(W_i [h_{t-1}, x_t])$	What new information to store
Cell Update	$c_t = f_t * c_{t-1} + i_t * \tanh(W_c [...])$	Update via addition prevents vanishing gradients
Output	$h_t = \text{sigmoid}(W_o [...]) * \tanh(c_t)$	Hidden state output for this timestep

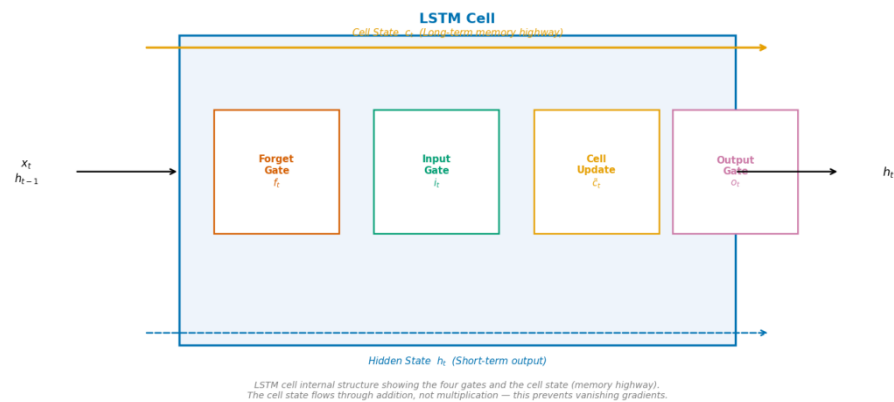


Figure 4: LSTM cell. The orange cell state flows through addition, preventing vanishing gradients

The training strategy for anomaly detection is to train the LSTM model only with normal data. Calculate MAE at test time between the predicted and actual readings. Normal = low error. Anomaly = high error. Threshold = $\text{mean}(\text{train_MAE}) + k \times \text{std}(\text{train_MAE})$, where the value of k is chosen.

Preprocessing

There are three decisions to prevent data leakage. (1) MinMax normalization on the training set only, and applied to the test set. (2) Split into anomaly-free time steps (70%) for training and the entire sequence for testing. The input is made up of 30 consecutive timesteps, and the 31st timestep is the target to be predicted. (3) Sliding window ($W=30$): 30 timesteps are used as inputs, and the 31st timestep is the target to be predicted as shown in Figure 5.

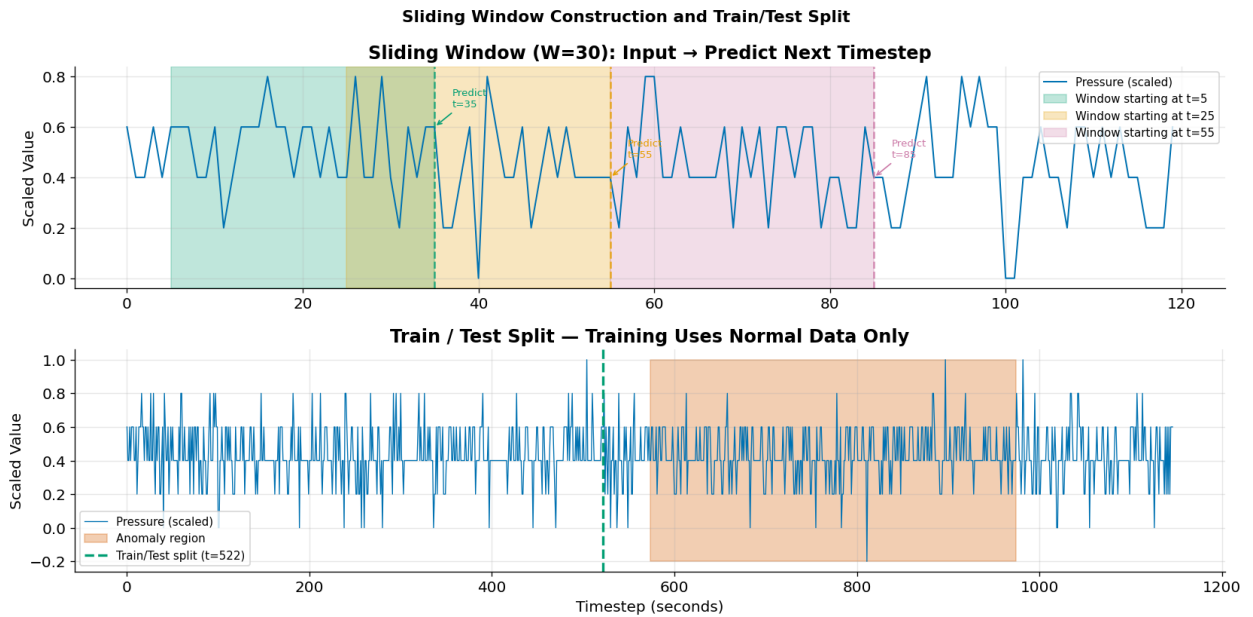


Figure 5: (Top) Overlapping windows $W=30$ predicting the next timestep. (Bottom) Train/test split, training uses only normal data

Model Architecture and Training

The stacked LSTM model (64 units followed by 32 units), with a dropout after every layer of 20%, was selected. The first LSTM layer generates a hidden state at each time step to be given to the second LSTM layer, which reduces the sequence to a single context vector.

Table 3: LSTM model architecture

Layer	Type	Units	Key Setting	Purpose
1	LSTM	64	return_sequences=True	Outputs a hidden state at every timestep for layer 2
2	Dropout	—	rate=0.20	Prevents over-reliance on specific neurons (Srivastava et al., 2014)
3	LSTM	32	return_sequences=False	Compresses the sequence to a single context vector
4	Dropout	—	rate=0.20	Second regularization stage
5	Dense	8	Linear activation	Predicts next timestep for all 8 sensor channels

Training Results

A Dense (8) output predicts all 8 sensors at once. The training was stopped at epoch 25 by the Adam optimizer (Kingma and Ba, 2014), MSE loss, and Early Stopping (patience=10).

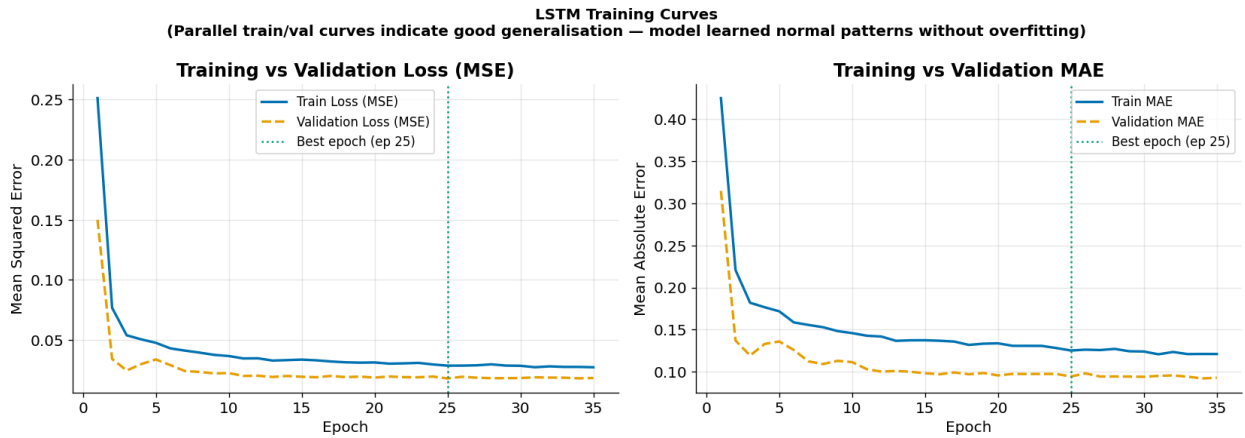


Figure 6: Training and validation MSE (left) and MAE (right). Parallel curves confirm good generalization. Best weights at epoch 25

The model learned real normal patterns as shown in Figure 6 by parallel train and validation curves. If the curves go outwards, it would indicate overfitting the training noise, instead of generalizing to new data.

Anomaly Detection via Prediction Error

The trained LSTM is applied to the entire test sequence. The anomaly score at each timestep is the MAE of the 8 sensor channel predictions and actual reading. Only the training errors are used to compute the threshold, namely $\text{threshold} = \text{mean}(\text{train_MAE}) + 3.0 \times \text{std}(\text{train_MAE}) = 0.1990$. Any time step larger than this is set as an anomalous one.

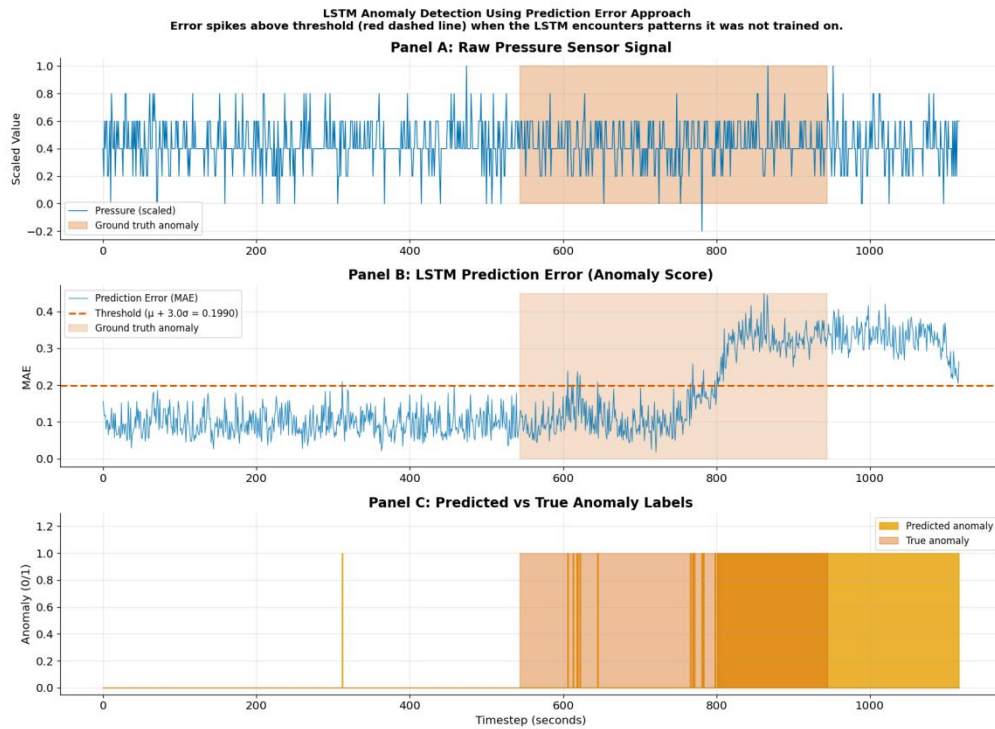


Figure 7: (A) Raw pressure signal. (B) Prediction error rises from $t=570$, exceeds threshold from $t=750$. (C) Predicted vs true labels

The total error at the peak anomaly ($t=862$) is broken down into Figure 8. Physically meaningful and dominating are accelerometers (errors 0.879, 0.839), where the vibration is directly proportional to the shaft imbalance. The least contribution is from temperature (0.016) because of the thermal inertia. This per-sensor breakdown is useful for providing diagnostic information in addition to the anomaly flag.

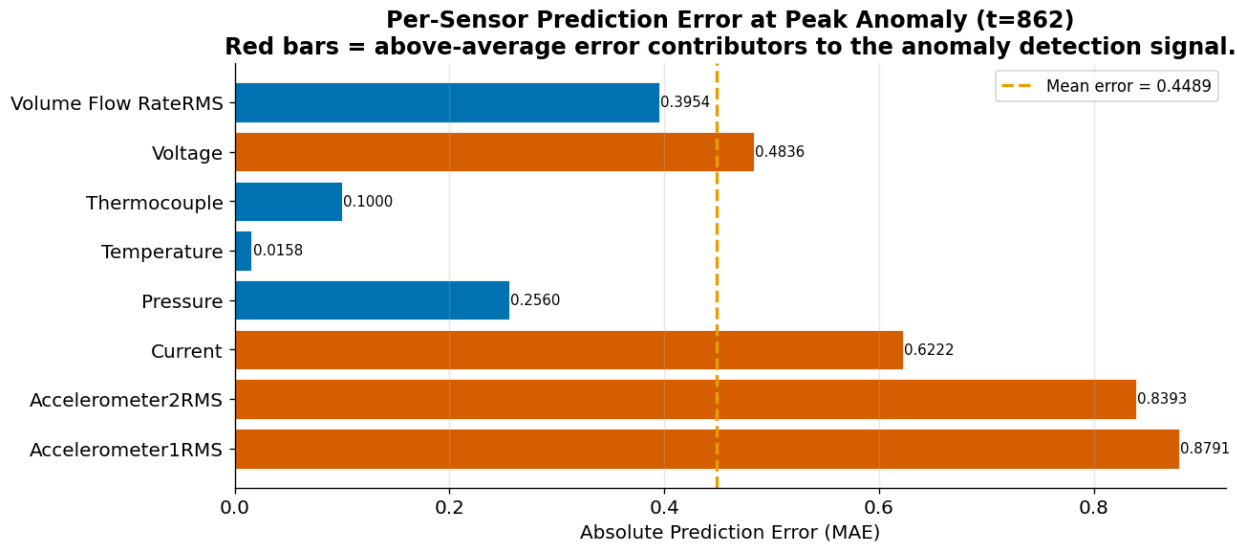


Figure 8: Per-sensor error at $t=862$. Accelerometers drive the signal; temperature is least informative

Results and Critical Analysis

For anomaly detection, standard accuracy is misleading because if an anomaly is always predicted as normal, it will have >60% accuracy even though it is predicting zero faults. There are three metrics:

Table 4: Evaluation metrics at default threshold $k=3$

Metric	Formula	Score	Meaning
Precision	$TP/(TP+FP)$	0.473	47% of flagged timesteps were real faults
Recall	$TP/(TP+FN)$	0.387	39% of true fault timesteps detected
F1-Score	$2PR/(P+R)$	0.425	Harmonic mean balanced summary

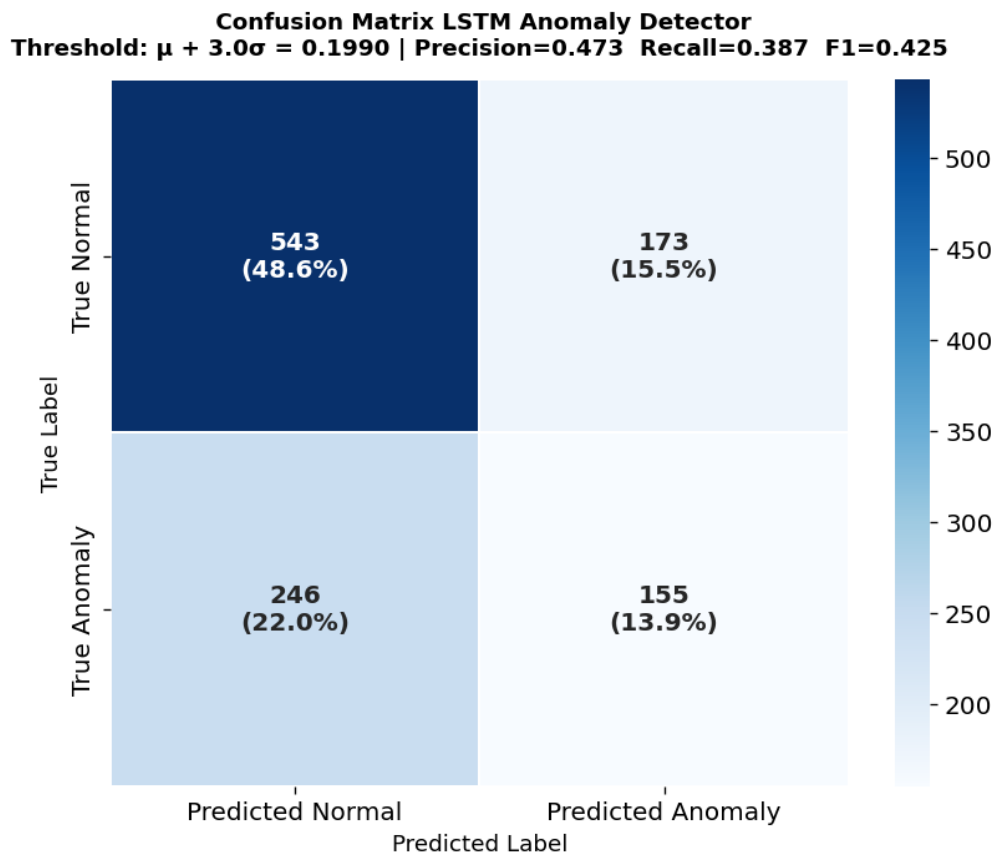


Figure 9: Confusion matrix 155 TP, 246 FN, 173 FP, 543 TN

Why these results? Three specific causes

The medium F1 score of 0.425 is not a failure of the approach; it is an informative result that tells you there are certain limits, and they are explainable. Three factors contribute:

Gradual fault onset (false negatives): The fault onset is gradual (from $t=570$), but the model is able to detect the fault reliably only from $t=750$. As the fault develops, the error increases gradually, until the fault becomes threshold. This is a natural consequence of the prediction-based approaches on gradual faults.

Normal pressure spikes (false positives). During a normal pressure spike, one false alarm occurs near $t=300$. During normal operation, pressure varies and can be a source of false alarms. This could be mitigated by using a per-sensor threshold.

Single-experiment training: Training was performed on one experiment file (data file). Considering training in all 35 SKAB normal files would provide a more comprehensive normal representation and better recall.

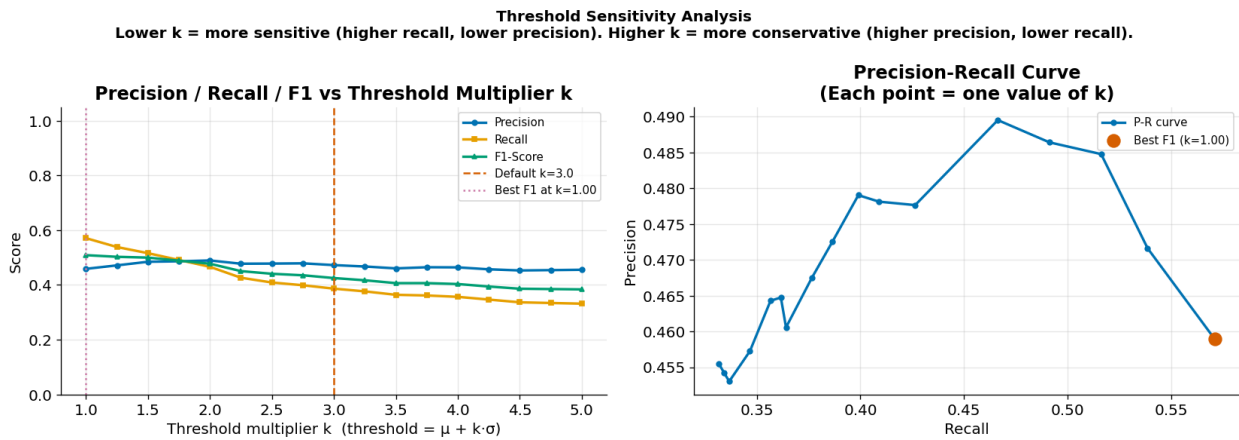


Figure 10: (Left) Precision, Recall, and F1 vs threshold multiplier k . Best F1 occurs at $k=1.0$. (Right) Precision-Recall curve each point represents one value of k . The curve reveals the fundamental tradeoff.

In Figure 10, it can be seen that the default $k=3$ is not optimal. A lower K value (such as 1.0) will increase recall at the expense of increasing the number of false positives captured during the early anomaly phase. In an industrial safety context, this balance is a major advantage to have a lower k because if a real fault occurs, it could be fatal, but if a false alarm occurs, it would result in an unnecessary inspection that would only be costly. The best value of " k " will vary based on the risk tolerance and cost structure of each application, and not on some universal mathematical principle.

Practical Recommendations and Ethics

Recommendations

1. Anomaly windows are known as event windows because they destroy the prediction-error signal by training on confirmed normal data.

2. Only leakage occurs with test data when all parameters (scaler, threshold) are fitted from the training data.
3. Lower k: More false alarms and detects early anomalies.
4. Use per-sensor error for diagnostics aggregate MAE flags anomalies; decomposition identifies which subsystem is failing.
5. Periodically retrain the system, as system behavior is changing; a static model is producing false positives.

Ethical Considerations

1. **Worker safety and accountability:** If the system fails to pick up the fault, no one is accountable. Restrictions should be explicitly documented before deployment to ensure safety-critical considerations are addressed.
2. **Over-reliance and deskilling:** People who rely on the automated detection can lose the skill of manual detection. These systems should be used in addition to, not in place of, human expertise.
3. **Workplace privacy:** Continuous sensor monitoring of operator-specific equipment may be indirect worker surveillance as per principle. Data governance and consent policies need to be implemented.
4. **Fairness across contexts:** A model based on well-maintained equipment might not work on older equipment, putting two safety standards at risk. Validation over a wide range of environments is a must.

Conclusion

This tutorial has demonstrated an end-to-end LSTM-based anomaly detection pipeline on real industrial sensor data. The key contribution is not merely a working model, but an honest account of what the model does, why it works, and precisely where and why it fails.

The pipeline normalizes the training data, creates sliding windows, trains an LSTM on normal windows, and thresholds the prediction error directly to any multivariate time-series anomaly detection problem, like server telemetry, electrocardiogram monitoring, power grid fault detection, or financial transaction streams. Confusion matrices and decomposing per-sensor errors and sweeping the threshold sensitivity have been considered not only relevant, but crucial analytical concepts for all.

Reference

- Hochreiter, S., Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780.
- Katser, I.D., Kozitsin, V.O. (2020). SKAB. Kaggle. doi.org/10.34740/KAGGLE/DSV/1693952
- Malhotra, P. et al. (2015). LSTM Networks for Anomaly Detection in Time Series. *ESANN* 2015.
- Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning Publications.
- Kingma, D.P., Ba, J. (2014). Adam: A Method for Stochastic Optimization. arXiv:1412.6980.
- Srivastava, N. et al. (2014). Dropout. *JMLR*, 15, 1929-1958.
- Keras Documentation (2024). <https://keras.io/>
- Scikit-learn Documentation (2024). <https://scikit-learn.org/>